



I 77131

2019-2020

## Examen JEE

ISCAE – 3<sup>ème</sup> année (IG, ID, RT)

Documents autorisés

I – GIT, Web Service Rest et Servlet

| Propositions                                                                                                                            | Réponse    |  |
|-----------------------------------------------------------------------------------------------------------------------------------------|------------|--|
| 1. Quels sont les langages WEB qu'un navigateur est capable d'interpréter/exécuter ?                                                    | PHP        |  |
|                                                                                                                                         | JavaScript |  |
|                                                                                                                                         | CSS        |  |
|                                                                                                                                         | JSP        |  |
|                                                                                                                                         | HTML       |  |
| 2. Après avoir créé un repository GIT, est-il nécessaire d'exécuter la commande <i>git init</i> avant chaque utilisation de celui-ci ?  | Oui        |  |
|                                                                                                                                         | Non        |  |
| 3. Est-ce que la création d'un repository local via la commande <i>git init</i> suffit pour créer le repository distant ?               | Oui        |  |
|                                                                                                                                         | Non        |  |
| 4. L'exécution de la commande <i>git status</i> est-elle obligatoire avant de faire les commandes <i>git add</i> et <i>git commit</i> ? | Oui        |  |
|                                                                                                                                         | Non        |  |
| 5. Citez deux interfaces Web (outils) qui permettent de gérer des repositories de Git sur internet (Gratuite ou payante).               |            |  |
| 6. A quoi sert JMS                                                                                                                      |            |  |

1. Quels sont les différentes familles de code http. Citez les et indiquer à quoi sert chaque famille (200 ...)

### II – Git, Web Service Rest et Servlet

#### 1<sup>ère</sup> partie

Nous avons la structure de donnée suivante permettant de gérer des utilisateurs

```
Map<String, User> users = new HashMap<>();
```

الاتحاد الوطني لطلبة  
U.N.E.M. موريتانيا

"Suite"

JEE

2019-2020

```
public class User {
    private String id;
    private String name;
    private String passwd;

    public User(String id, String name, String passwd) {
        this.name = name;
        this.id = id;
        this.passwd = passwd;
    }
}
```

Pour ajouter un utilisateur dans cette structure, il suffit de donner son identifiant et un Objet de type User. Exemple : `users.put("1234", new User("1234", "Ahmed", "Ahmed"))` ;

La suppression se fait via la méthode « `remove` ». Par exemple `users.remove("1234")` supprime de la structure `users` l'utilisateur dont l'identifiant "1234".

L'objectif de cet exercice est de créer une classe `UserController` qui expose des services Web REST permettant de faire les tâches suivantes :

1. Ajouter un utilisateur
2. Modifier le mot de passe d'un utilisateur (champs `passwd`)
3. Supprimer un utilisateur
4. Afficher la liste de tous les utilisateurs
5. Afficher un utilisateur donné grâce à son identifiant

Penser à bien apporter les modifications nécessaires à la classe `User` ainsi qu'à la classe `UserController`

Ces étapes doivent être effectuées tout en respectant les bonnes pratiques du développement logiciel (Utilisation de Git).

2<sup>ème</sup> partie

On veut maintenant ajouter le contrôle suivant. Avant qu'un utilisateur modifie son mot de passe, l'application s'assure que son mot de passe actuel correspond bien à celui déjà stocké dans l'objet `user`.

1. Quel est le meilleur endroit pour effectuer cette vérification de mot de passe ?
2. Décrire les étapes à faire pour ajouter cette vérification de mot de passe (Création nouvelle classe, modification du fichier `web.xml`...)
3. Ecrire le code nécessaire permettant de rendre obligatoire la vérification du mot de passe avant de le modifier.



## Solution Examen JEE 2019-2020

### I. Git

1. Html, CSS, Js
2. Non
3. Oui (git init)
4. Oui
5. Gitalist, GitLab
6. Java Message Service (JMS) permet d'envoyer et de recevoir des messages de manière asynchrone entre applications ou composants java.
7. Les différentes familles de code http
  - 200 : succès de la requête
  - 301 et 302 : redirection, respectivement permanente et temporaire
  - 401 : utilisateur non authentifié
  - 403 : accès refuse
  - 404 : page non trouvée
  - 500 et 503 : erreur serveur
  - 504 : le serveur n'a pas répondu

### II. Partie

```
package ExamenJee.services;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;

import javax.ws.rs.Consumes;
import javax.ws.rs.POST;
import javax.ws.rs.GET;
import javax.ws.rs.DELETE;
import javax.ws.rs.PUT;
import javax.ws.rs.Path;
import javax.ws.rs.QueryParam;
import javax.ws.rs.PathParam;
import javax.ws.rs.core.MediaType;

import ExamenJee.modeles.DataUser;

public class ControllerUser {

    private static Map<String, User> Data_User = Data.getDataUser();

    // 1) Cette Methode pour ajouter un utilisateur
    public User ajoutUtilisateur(String id, User newUser) {

        Data_User.put(newUser.getId(), newUser);
        return newUser;
    }

    @POST
    @Consumes(MediaType.APPLICATION_JSON) //Indique le type de donnees accepte en
    entree par le web service
    @Path("/ajouterUtilisateur")
    public User AjoutUtilisateur(@QueryParam("id") String id, User newUser) {
        return ajoutUtilisateur(id, newUser);
    }
}
```



```
}
// 1) Cette Methode pour afficher la liste des utilisateurs
// 2) Cette Methode pour change la Mot de passe de l'utilisateur
public String changePass(String id, String Nmotpass, String MotPasse) {
    User user = Data_User.get(id);
    if (user == null) {
        return "Cette utilisateur n'existe pas !!";
    } else if (!MotPasse.equals(user.getPassword())) {
        return "Mot de passe incorrecte !!";
    } else {
        user.setPassword(Nmotpass);
        Data_User.put(user.getId(), user);
        return "Mot de passe est modifié avec succès";
    }
}

@PUT
@Path("/ChangePassword/{id}/{Password}/{NewPassword}")
public String ChangePassword(@PathParam("id") String id,
    @PathParam("Password") String Password,
    @PathParam("NewPassword") String NewPassword) {
    return changePass(id, Password, NewPassword);
}
// Fin 2
// 3) Cette Methode pour supprimer un utilisateur
public String supprimerUtilisateur(String id, String name) {
    User user = Data_User.get(id);
    if (user == null) {
        return "l'utilisateur n'existe pas";
    }
    Data_User.remove(id);
    return "l'utilisateur est supprime avec succee";
}
@DELETE
@Consumes(MediaType.APPLICATION_JSON)
@Path("/supprimerUtilisateur")
public String supp_Utilisateur(@QueryParam("id") String id, @QueryParam("name")
    String name) {
    return supprimerUtilisateur(id, name);
}
// Fin 3

// 4) Cette Methode GET, affiche les listes des utilisateur
public List<User> afficherListUtilisateurs() {
    return new ArrayList<User>(Data_User.values());
}
@GET
@Consumes(MediaType.APPLICATION_JSON)
@Path("/afficherListUtilisateur")
public List<User> afficherListUtilisateur() {
    return afficherListUtilisateurs();
}
// Fin 4
}
```